

Developer feedback

Octro — XRPL Lending Protocol Hackathon, Paris, 12–13 September 2026

TRACK	1 — open-ended vault	NETWORK	Custom Hackathon Devnet
FLAVOUR	Loaded — Credentials, Domains	NETWORK ID	4001
PROTOCOL	Lending Protocol V1	RIPPLED	3.4.0-rc1
LIBRARY	xrpl@5.2.0 (stable), Node v22.23.1	TEAM	Kevin, Samet, Augustin

All observations below come from real work sessions on 12 September 2026. Each one has a transaction hash, or a recorded absence of one. The mandatory DevEx hook captured the retry loops. We kept only the items that we could confirm against the ledger. We report three frictions, because these three cost the most time and can hit every Track 1 team.

FRICION 1

CATEGORY: sdk / SEVERITY: high

A new account can create a dead transaction, and the network reports no error

What we did. We funded a new account through the event faucet. We then built a `PermissionedDomainSet` with `autofill` and submitted it at once.

Expected. The transaction validates, or it fails with a result code.

Observed. `submit` answered `engine_result: tesSUCCESS`. The transaction never validated. It stayed unknown to the ledger for minutes, with a large `LastLedgerSequence` margin. There is no hash to show, because nothing was ever included. This is the point: the failure leaves no trace.

Cause. `autofill` read the `Sequence` with `account_info` before the faucet's own funding `Payment` was validated. It therefore chose a `Sequence` one below the real starting value. The account can never use that value. We confirmed the real starting `Sequence` with `account_tx`.

Repro. Fund an account through <https://lending-hackathon-faucet.dev.ripple.io/accounts>. Immediately call `autofill` and `submit` for any transaction from that account. Do not wait for the funding payment.

Cost. About 15 minutes, plus a debug script. The hook recorded two `retry_resolved` events, at 14:08 and 14:14.

Proposed fix. Make `fundwallet()` wait for its own funding transaction to validate before it returns. Make `autofill` refuse to guess a `Sequence` for an account that is not yet on the validated ledger, instead of guessing one. A refusal with a clear message is better than a silent dead transaction. Our workaround is to poll `account_info` with `ledger_index: 'validated'` until it answers.

FRICION 2

CATEGORY: client libraries / SEVERITY: high

LoanSet needs the broker owner's signature, but `temBAD_SIGNER` does not say so

What we did. We created a vault, deposited capital and set a loan broker. All three validated. We then submitted a `LoanSet` signed by the borrower only, with a valid `LoanBrokerID` and a valid `PrincipalRequested`.

Expected. The transaction reaches consensus. If it fails, it fails for a financial reason, for example insufficient cover.

Observed. The node refused it before inclusion with `temBAD_SIGNER`. The response names no field and no missing party. Rejected attempt: `B9A12BE23AAF37771569A6B46AD22A964E4A9D35D6157D06A05AEA1C39A75447`.

Cause. `LoanSet` is a two-party transaction. It needs the loan broker owner's `CounterpartySignature`. The name `temBAD_SIGNER` suggests a wrong signature, not a missing second party. The SDK's runtime warning speaks about the fee for the counterparty's signers. It does not say that a signature is missing.

How we solved it. We read `loanSet.d.ts` in the installed package. We found the `Counterparty` and `CounterpartySignature` fields, and the helpers `signLoanSetByCounterparty` and `combineLoanSetCounterpartySigners`. The borrower signs with `wallet.sign()`. The broker owner then

co-signs the same `tx_blob`. For one counterparty signer, no combine step is necessary. Result: 693C846FF98C75E74FDF147E549500101320EDF1A193E95E19592660929B6FE2 (ledger 65045).

Cost. About 20 minutes. We think every Track 1 team meets this before it reads the SDK source.

Proposed fix. Add a specific code, for example `temMISSING_COUNTERPARTY_SIG`, or name the missing field in the error message. Publish one worked example of the two-party signature in the Lending Protocol documentation. The SDK already ships the helpers; the documentation does not show them together.

FRICION 3

CATEGORY: missing primitive / SEVERITY: high

`tfLoanFullPayment` asks for an amount that a client cannot express

What we did. We repaid a loan in full. We read `TotalValueOutstanding` from the Loan ledger entry, which showed 10001370 drops. We submitted a `LoanPay` with that Amount and Flags: `tfLoanFullPayment`.

Expected. The loan closes.

Observed. The transaction entered a validated ledger, then rolled back with `tecKILLED`, "No funds transferred and no offer created". The network still charged the fee. Attempt: 28343EEE820EB27EEC7A1B5CC02E22005E6CD87E37E05365BF85ED3869B546BD (ledger 65064).

Cause. The real payoff is the unrounded `PeriodicPayment`, which the ledger holds as 10001369.86301369963. That value is not an integer number of drops. The flag appears to demand an exact match. The ledger shows the rounded figure, so the exact figure is not available to the client. We repaid with the same Amount and Flags: 0, and the loan closed: EA2B4816AE06E08DA64FD8C6131C0410691951D3350A21A387ED092334A3C1F1.

Why this matters. The values that the ledger publishes cannot satisfy the flag. A developer can only find this by trial and error, and each trial costs a fee.

Proposed fix. Publish an exact `PayoffAmount` in the loan object, in a form that a client can submit. Or let `tfLoanFullPayment` ignore Amount and settle the true balance. Or round the required amount in the borrower's favour. Also make `tecKILLED` name the expected amount.

Other observations, in short

The faucet breaks `fundWallet()`. It answers with an account object that holds address and secret. The helper does not parse that shape. It throws `XRPLFaucetError: The faucet account is undefined, which reads like an outage.`

The SDK's own minimum is below what rippled accepts. `validateLoanSet` enforces `MIN_PAYMENT_INTERVAL = 60`. The node then refuses `PaymentInterval: 60` with a generic `temINVALID` that names no field. 3600 works.

`submitAndWait` fails with a message that contradicts itself on this fast devnet: "...greater than the transaction's `LastLedgerSequence` ...Preliminary result: `tesSUCCESS`".

`VaultCreate.DomainID` is not on the vault. It is stored on the share `MPTokenIssuance`, at `vault_info.shares.DomainID`. We found it by dumping the full response.

`VaultSet` with a `DomainID` needs a vault that is already private. A public vault answers `tecNO_PERMISSION`, which names no reason: 6C50EA54AC253A56.... The same call on a private vault succeeded.

Fix: return the standard shape, or publish the parser in the event README. We POST to the faucet directly and build the wallet with `wallet.fromSeed()`.

Fix: align the SDK constant with the deployed rule, and name the rejected field in `temINVALID`. Cost us about 15 minutes and one full rebuild of a loan scenario.

Fix: derive the default `LastLedgerSequence` margin from the observed close cadence, and reword the message.

Fix: document this, and mirror `DomainID` at the top level of `vault_info`. Domain-gating a vault really gates who may hold its share MPT.

Fix: document the pre-condition, or allow the transition, or return a code that names it.

DIDSet has an undocumented maximum length. A full W3C document gives `temMALFORMED`. A short JSON object works.

The DevEx hook records false transactions. Its text matcher read our own Python constant `INFEASIBLE_NO_DEBT` as a result code, and a grep over `.ts` imports as a `Payment`. Its duplicate check uses the text only, so a reflection sent with a wrong `session_id` cannot be sent again under the correct one.

The amendment set was not announced. This ledger enables `LendingProtocol` and `LendingProtocolV1_1` together. The event page warns that V1.1 restricts new loans to closed-ended vaults.

Fix: publish the limit for `DIDDocument` and `URI`.

Fix: accept a match only from output that looks like a submit response, or only near a transaction hash. Key the duplicate check on the text and the `session_id` together.

Fix: state the final amendment set in writing before the event, and show the feature command in the quickstart.

What worked well

Credentials and Permissioned Domains behaved exactly as their names suggest. The deny, accept and deny-after-expiry cycle gave legible codes, `tecNO_AUTH` and `tecEXPIRED`, on the first attempt. Native fee sponsorship worked at once with `addPreFundedSponsor` and `signAsSponsor`; the balance deltas confirm that the sponsor paid the fee and the sponsee paid none. DID replay protection refused the wrong network with `telWRONG_NETWORK` and the wrong signer with `tefBAD_AUTH`. `VaultCreate`, `VaultDeposit`, `LoanBrokerSet` and `VaultWithdraw` all validated on the first correct submission.

Full transaction records, with hashes and explorer links: docs/progress/augustin/evidence/. Session captures: [.xrpl-devex/reports/](https://xrpl-devex/reports/).